Übungsblatt 7: Moderne CPU-Architekturen

Mikrocomputertechnik 3 (MCT3)

Zielsetzung

Sie vertiefen Vorlesung 7: Datenabhängigkeiten und Superskalarität, Register Renaming / ROB, Branch Prediction und Spekulation (inkl. Spectre) sowie Compiler-Optimierungen in Godbolt (`-O1` vs. `-O3`).

Bearbeitungszeit: ca. 45–60 Min. | **Orientierungspunkte:** Aufgabe 1: 8 P, Aufgabe 2: 12 P, Aufgabe 3: 4 P (Gesamt: 24 P)

Teil 1: Theorie — Von In-Order zu Out-of-Order

Aufgabe 1.1: Datenabhängigkeiten und Superskalarität

Superskalare Prozessoren nutzen Instruction-Level Parallelism (ILP) über mehrere Execution Units. Das größte Hindernis sind Datenabhängigkeiten.

Gegeben:

```
1: lw  t0, 0(s1)
2: add t1, t0, t2
3: sub t0, t4, t5
4: and t6, t0, t3
```

1. Identifizieren Sie eine *True Dependency* (Read-After-Write / RAW). Zwischen welchen Befehlen tritt sie auf, und warum kann die Hardware sie nicht parallelisieren?
2. Zwischen den Befehlen 2 und 3 liegt eine „falsche“ Abhängigkeit (Write-After-Read / WAR). Warum? Welches fatale Problem entstünde, wenn eine einfache CPU Befehl 3 vor Befehl 2 ausführen würde?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 1.2: Register Renaming und der ROB

Out-of-Order-(OoO-)CPUs lösen falsche Abhängigkeiten und führen Befehle asynchron aus.

1. Erklären Sie *Register Renaming*. Wie zerstört die Technik die WAR-Abhängigkeit zwischen Befehl 2 und 3 (Architektur- vs. physische Register)?
2. Warum sind Reorder Buffer (ROB) und *In-Order Commit* zwingend, um das System bei Interrupts vor inkonsistenten Zuständen zu schützen?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Teil 2: Theorie — Branch Prediction und Spekulation

Aufgabe 2.1: Das Problem tiefer Pipelines

Moderne Prozessoren haben oft Pipelines mit weit über 15 Stufen.

1. Erklären Sie das *Control-Hazard*-Problem bei Sprüngen (`beq`). Warum führen Fehlprognosen bei tiefen Pipelines zu massiven Leistungseinbrüchen (*Misprediction Penalty*)?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 2.2: Branch History und Spekulation

1. Skizzieren Sie einen *2-Bit saturierenden Zähler*. Welchen Vorteil hat er gegenüber einem 1-Bit-Prädiktor bei verschachtelten Schleifen (Hysterese)?
2. Erklären Sie *spekulative Ausführung*. Was tut die CPU, während sie auf die ALU-Bestätigung des Sprungs wartet?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 2.2b: 2-Bit-Prädiktor — Zustandstabelle

Ein 2-Bit saturierender Zähler startet im Zustand `11` (*Strongly Taken*). Kodierung: `00` Strongly NT, `01` Weakly NT, `10` Weakly T, `11` Strongly T. Vorhersage: Zustände `1x` → Taken, `0x` → Not Taken.

Gegeben die tatsächliche Sprungsequenz: **T, T, NT, NT, T**.

1. Füllen Sie eine Zustandstabelle: für jeden Schritt Startzustand, Vorhersage, tatsächliches Ergebnis, Treffer/Miss, Folgezustand.
2. Wie viele Treffer (korrekte Vorhersagen) ergeben sich insgesamt?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 2.3: Seitenkanalangriffe (Spectre)

1. Bei Misprediction macht der ROB einen *Rollback* (Squash). Welcher Zustand wird dabei typisch *nicht* gelöscht und öffnet die Tür für Spectre?
2. Wie nutzt ein Angreifer Timing als Orakel, um eigentlich unerreichbare Daten zu rekonstruieren?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Teil 3: Praxis — Performance-Analyse in Godbolt

Szenario: Der Compiler kann Code für die Pipeline vorbereiten. Nutzen Sie Compiler Explorer (godbolt.org).

Gegeben:

```
void vector_add(int *a, int *b, int *c, int len) {
    for (int i = 0; i < len; i++) {
        c[i] = a[i] + b[i];
    }
}
```

Aufgabe 3.1: Instruction Scheduling und Loop Unrolling

Öffnen Sie Godbolt, Compiler **RISC-V rv32gc gcc**, Code einfügen.

1. Kompilieren Sie mit `-O1`. Wo liegen typisch die `lw`- und `add`-Instruktionen in der Schleife?
2. Wechseln Sie auf `-O3`. Beobachten Sie:
    - *Instruction Scheduling:* Werden `lw`/`sw` so umsortiert, dass Load-Use-Stalls gemildert werden?
    - *Loop Unrolling:* Warum wird die Assembler-Ausgabe länger — und warum hilft das der Branch Prediction?

Dokumentieren Sie Ihre Beobachtungen in wenigen Sätzen.

Ihre Lösung

(Notieren Sie Ihre Beobachtungen hier.)